



Magíster en Estadística, PUCV

Logistic Regression

Alejandro Veloz

Overview

1. Why least squares fails for classification.
2. The logistic function and the log-odds.
3. The Bernoulli model and its likelihood.
4. Maximum likelihood, the gradient, and convexity.
5. Fitting: gradient ascent and Newton/IRLS.
6. Separable data and why regularization is not optional.
7. Multiple classes: softmax and cross-entropy.

The problem

Same setup as before, but the target is now a **label**:

$$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N, \quad t_n \in \{0, 1\}$$

The problem

Same setup as before, but the target is now a **label**:

$$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N, \quad t_n \in \{0, 1\}$$

We want $P(t = 1 \mid \mathbf{x})$ — a **probability**, not just a label.

The problem

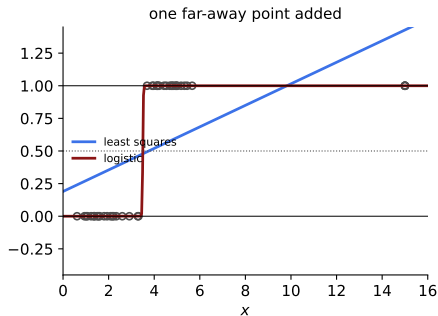
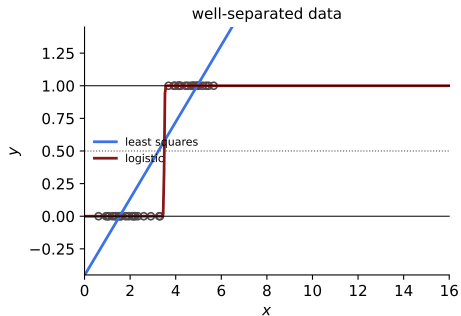
Same setup as before, but the target is now a **label**:

$$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N, \quad t_n \in \{0, 1\}$$

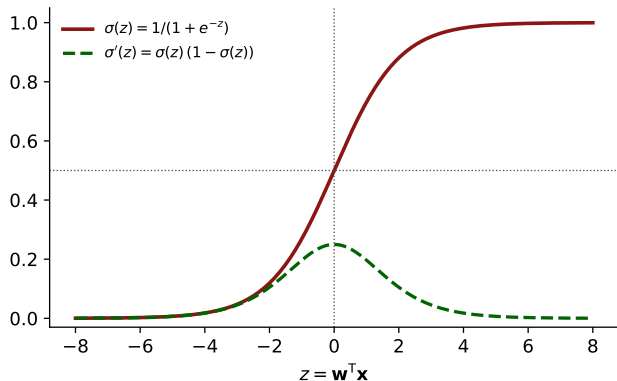
We want $P(t = 1 \mid \mathbf{x})$ — a **probability**, not just a label.

- Is this tumour malignant? And *how sure* are we?
- Will this patient be readmitted within 30 days?

Why not least squares?



The logistic function



$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma(-z) = 1 - \sigma(z), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

The model

$$P(t = 1 \mid \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

The model

$$P(t = 1 \mid \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

Rearranged, the model is **linear in the log-odds**:

$$\log \frac{P(t = 1 \mid \mathbf{x})}{P(t = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

The model

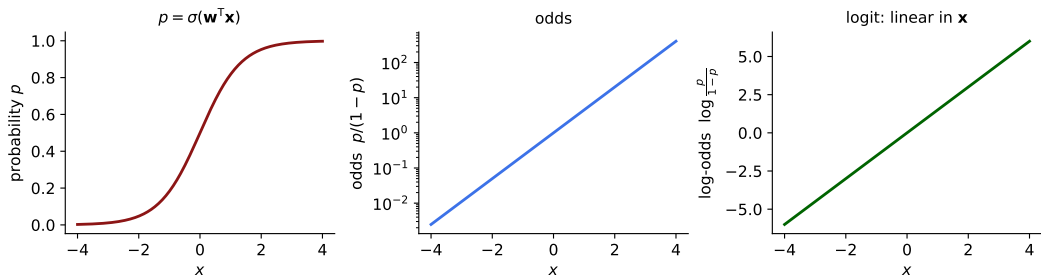
$$P(t = 1 \mid \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

Rearranged, the model is **linear in the log-odds**:

$$\log \frac{P(t = 1 \mid \mathbf{x})}{P(t = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

That is the definition of logistic regression, and it is where the word **linear** lives.

Probability, odds, log-odds



The same fitted model in three scales. Only the third is a straight line.

The Bernoulli model

For a single observation with $t \in \{0, 1\}$, write $y_n = \sigma(\mathbf{w}^\top \mathbf{x}_n)$:

$$p(t_n \mid \mathbf{x}_n, \mathbf{w}) = y_n^{t_n} (1 - y_n)^{1-t_n}$$

The Bernoulli model

For a single observation with $t \in \{0, 1\}$, write $y_n = \sigma(\mathbf{w}^\top \mathbf{x}_n)$:

$$p(t_n \mid \mathbf{x}_n, \mathbf{w}) = y_n^{t_n} (1 - y_n)^{1-t_n}$$

The exponents act as switches:

- if $t_n = 1$ this is y_n ,
- if $t_n = 0$ this is $1 - y_n$.

The likelihood

Assuming the observations are independent given the inputs,

$$p(\mathbf{t} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

The likelihood

Assuming the observations are independent given the inputs,

$$p(\mathbf{t} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

Take minus the log — the **cross-entropy** or **logistic loss**:

$$E(\mathbf{w}) = - \sum_{n=1}^N \left[t_n \ln y_n + (1 - t_n) \ln(1 - y_n) \right]$$

The likelihood

Assuming the observations are independent given the inputs,

$$p(\mathbf{t} \mid \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

Take minus the log — the **cross-entropy** or **logistic loss**:

$$E(\mathbf{w}) = - \sum_{n=1}^N \left[t_n \ln y_n + (1 - t_n) \ln(1 - y_n) \right]$$

Unlike least squares, there is **no closed-form solution**: y_n depends on $\mathbf{w}$ through σ .

The gradient

Using $\sigma' = \sigma(1 - \sigma)$, everything cancels:

$$\frac{\partial E}{\partial w_j} = - \sum_{n=1}^N \left(\frac{t_n}{y_n} - \frac{1 - t_n}{1 - y_n} \right) y_n (1 - y_n) x_{nj} = \sum_{n=1}^N (y_n - t_n) x_{nj}$$

The gradient

Using $\sigma' = \sigma(1 - \sigma)$, everything cancels:

$$\frac{\partial E}{\partial w_j} = - \sum_{n=1}^N \left(\frac{t_n}{y_n} - \frac{1 - t_n}{1 - y_n} \right) y_n (1 - y_n) x_{nj} = \sum_{n=1}^N (y_n - t_n) x_{nj}$$

$$\nabla E(\mathbf{w}) = \mathbf{X}^T (\mathbf{y} - \mathbf{t}) = \sum_{n=1}^N \underbrace{(y_n - t_n)}_{\text{prediction-label}} \mathbf{x}_n$$

The gradient

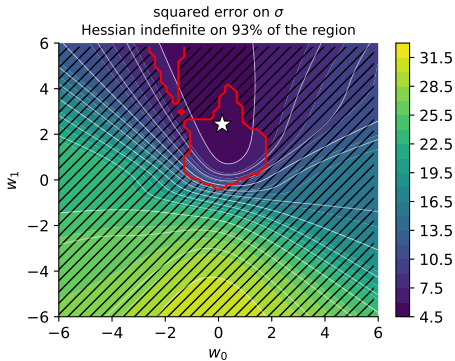
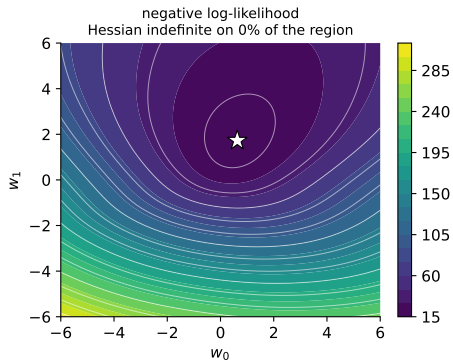
Using $\sigma' = \sigma(1 - \sigma)$, everything cancels:

$$\frac{\partial E}{\partial w_j} = - \sum_{n=1}^N \left(\frac{t_n}{y_n} - \frac{1 - t_n}{1 - y_n} \right) y_n (1 - y_n) x_{nj} = \sum_{n=1}^N (y_n - t_n) x_{nj}$$

$$\nabla E(\mathbf{w}) = \mathbf{X}^T (\mathbf{y} - \mathbf{t}) = \sum_{n=1}^N \underbrace{(y_n - t_n)}_{\text{prediction-label}} \mathbf{x}_n$$

Identical in form to the gradient of least squares. That is not a coincidence.

Convexity



$\nabla^2 E = X^T R X$ with $R_{nn} = y_n(1 - y_n) > 0$, so the Hessian is positive semi-definite.

Fitting: gradient ascent and Newton

Gradient descent. $w \leftarrow w - \alpha X^T(y - t)$. Cheap per step, many steps, needs a learning rate.

Fitting: gradient ascent and Newton

Gradient descent. $w \leftarrow w - \alpha X^T(y - t)$. Cheap per step, many steps, needs a learning rate.

Newton / IRLS (ESL §4.4.1). Use the Hessian $\nabla^2 E = X^T R X$:

$$w^{\text{new}} = w - (X^T R X)^{-1} X^T (y - t) = (X^T R X)^{-1} X^T R z$$

with the adjusted response $z = Xw + R^{-1}(t - y)$.

Fitting: gradient ascent and Newton

Gradient descent. $w \leftarrow w - \alpha X^T(y - t)$. Cheap per step, many steps, needs a learning rate.

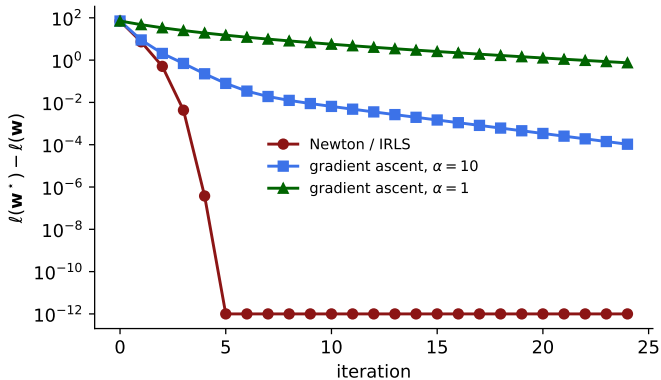
Newton / IRLS (ESL §4.4.1). Use the Hessian $\nabla^2 E = X^T R X$:

$$w^{\text{new}} = w - (X^T R X)^{-1} X^T (y - t) = (X^T R X)^{-1} X^T R z$$

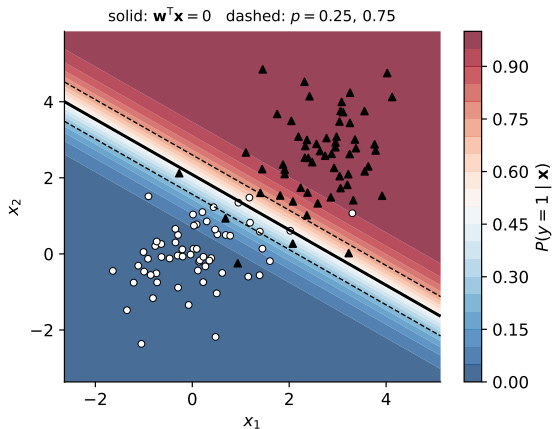
with the adjusted response $z = Xw + R^{-1}(t - y)$.

Each Newton step is a weighted least-squares fit — hence *iteratively reweighted least squares*.

Newton converges much faster

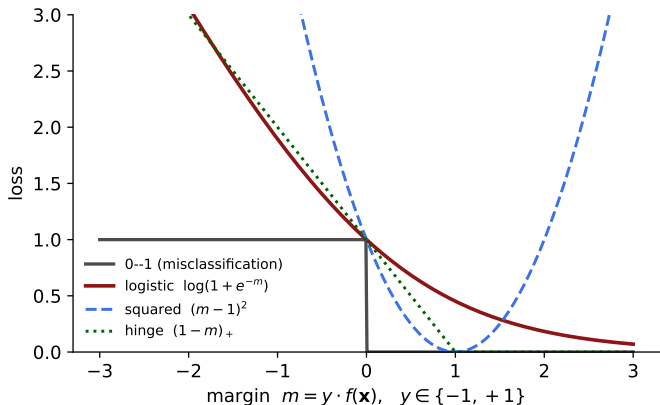


What the fitted model looks like



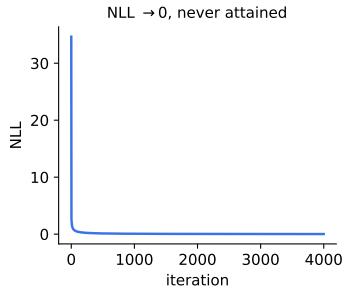
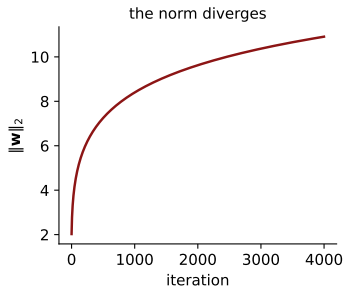
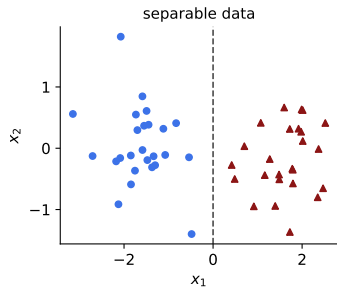
The boundary $\mathbf{w}^T \mathbf{x} = 0$ is a **hyperplane**; $\|\mathbf{w}\|$ controls how fast the probability turns.

The logistic loss among its relatives



With $y \in \{-1, +1\}$ and margin $m = y f(\mathbf{x})$, all of these are convex surrogates for the 0-1 loss.

Separable data breaks maximum likelihood



Regularized logistic regression

Add exactly the penalties from the previous lecture:

$$E(\mathbf{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] + \lambda \|\mathbf{w}\|_2^2 \text{ or } \lambda \|\mathbf{w}\|_1$$

Regularized logistic regression

Add exactly the penalties from the previous lecture:

$$E(\mathbf{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] + \lambda \|\mathbf{w}\|_2^2 \text{ or } \lambda \|\mathbf{w}\|_1$$

- The penalty makes the optimum **finite** even for separable data.
- L_2 = MAP with a Gaussian prior; L_1 = MAP with a Laplace prior, and it selects features.
- **Do not penalize the intercept, and standardize the inputs.**

Regularized logistic regression

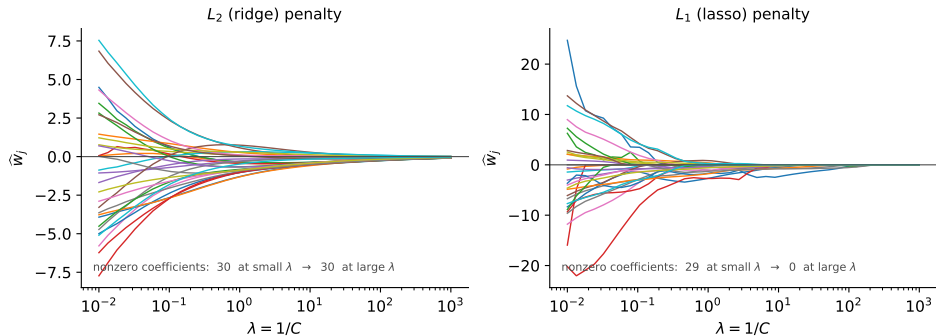
Add exactly the penalties from the previous lecture:

$$E(\mathbf{w}) = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] + \lambda \|\mathbf{w}\|_2^2 \text{ or } \lambda \|\mathbf{w}\|_1$$

- The penalty makes the optimum **finite** even for separable data.
- L_2 = MAP with a Gaussian prior; L_1 = MAP with a Laplace prior, and it selects features.
- **Do not penalize the intercept**, and **standardize the inputs**.

In sklearn the knob is $C = 1/\lambda$, and it is regularized *by default*.

Regularization paths



Breast cancer data, 30 standardized features. Same picture as ridge and lasso, different loss.

More than two classes: softmax

For $t \in \{1, \dots, K\}$, one weight vector per class and a normalized exponential:

$$P(t = k \mid \mathbf{x}, W) = \frac{\exp(\mathbf{w}_k^T \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \mathbf{x})}$$

More than two classes: softmax

For $t \in \{1, \dots, K\}$, one weight vector per class and a normalized exponential:

$$P(t = k \mid \mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Negative log-likelihood with one-hot targets t_{nk} — the **cross-entropy**:

$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}, \quad \frac{\partial E}{\partial \mathbf{w}_k} = \sum_{n=1}^N (y_{nk} - t_{nk}) \mathbf{x}_n$$

More than two classes: softmax

For $t \in \{1, \dots, K\}$, one weight vector per class and a normalized exponential:

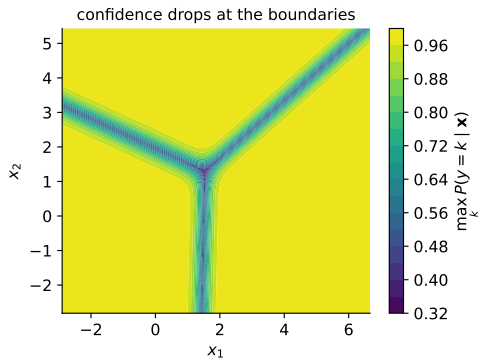
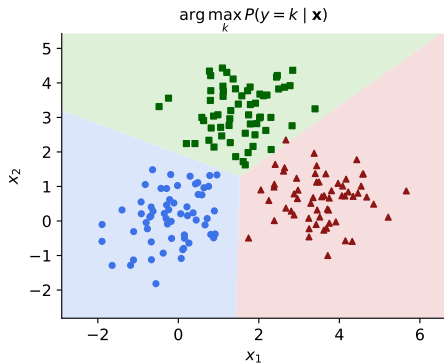
$$P(t = k \mid \mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Negative log-likelihood with one-hot targets t_{nk} — the **cross-entropy**:

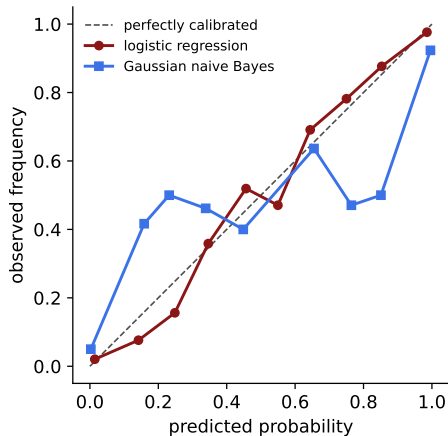
$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}, \quad \frac{\partial E}{\partial \mathbf{w}_k} = \sum_{n=1}^N (y_{nk} - t_{nk}) \mathbf{x}_n$$

The same gradient again: (prediction — label) $\times$ feature.

Softmax in two dimensions



Are the probabilities honest?



Of the cases predicted at 0.7, do about 70% actually turn out positive?

Summary

- Same recipe as lecture 1: Bernoulli likelihood $\Rightarrow$ **logistic loss**.
- The model is **linear in the log-odds**; e^{w_j} is an odds ratio.
- Gradient is $X^T(y - t) - (\text{prediction} - \text{label}) \times \text{feature}$, as in least squares and softmax.
- The NLL is **convex**; Newton/IRLS solves it as repeated weighted least squares.
- Separable data $\Rightarrow$ the ML estimate **does not exist**. Regularize; it is not optional.
- The output is a **probability**: check that it is calibrated, not just accurate.